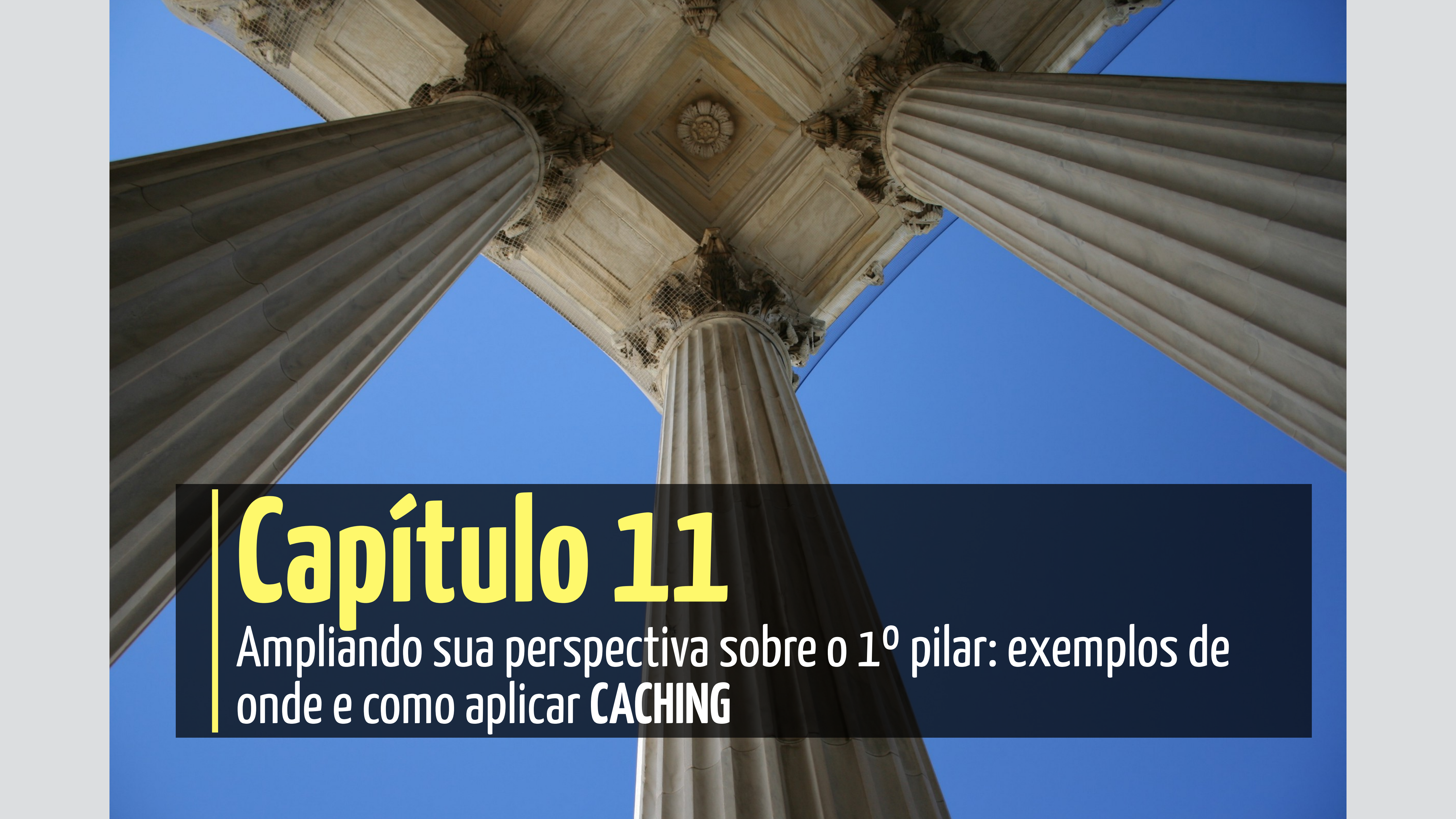


A low-angle, upward-looking photograph of several large, fluted classical columns. The columns are made of light-colored stone or marble and are topped with ornate capitals. They converge towards the top of the frame, creating a strong sense of height and grandeur. The background is a clear, bright blue sky. The overall composition is symmetrical and emphasizes the architectural details and scale of the structure.

Capítulo 11

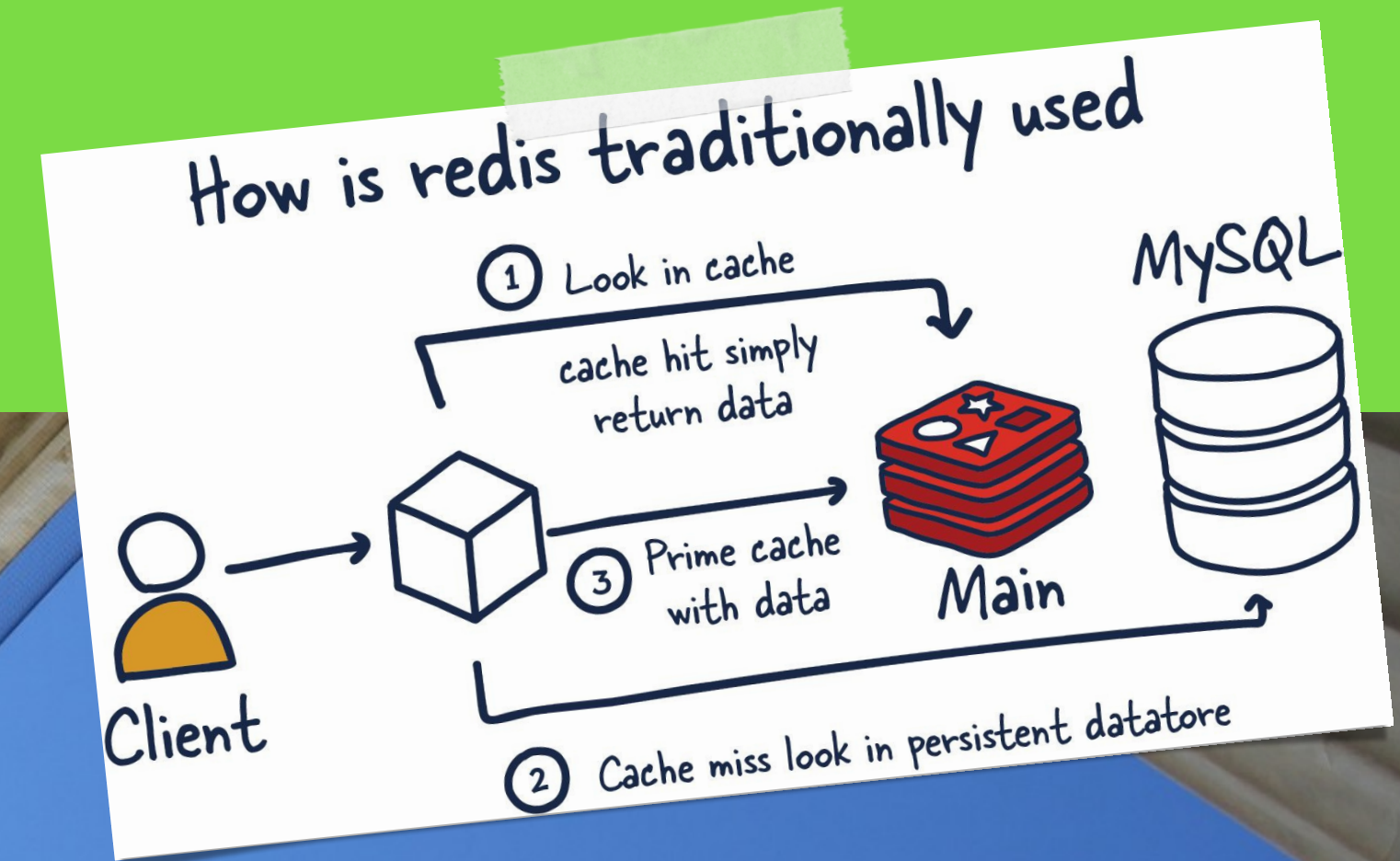
Ampliando sua perspectiva sobre o 1º pilar: exemplos de onde e como aplicar CACHING



CACHE



CACHE

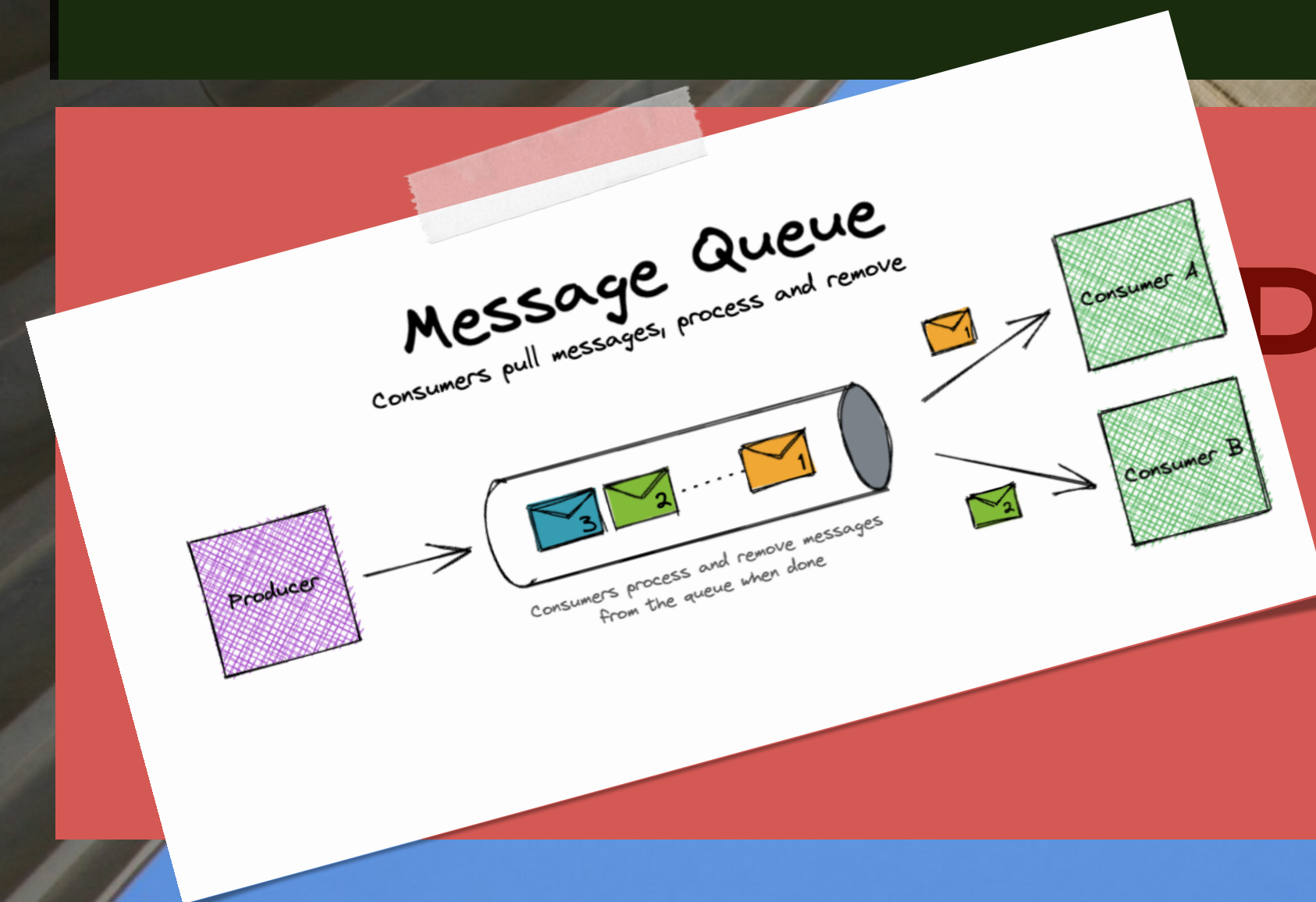




CACHE

PROCESSAMENTO
ASSINCRONO

CACHE



PROCESSAMENTO ASSINCRONO

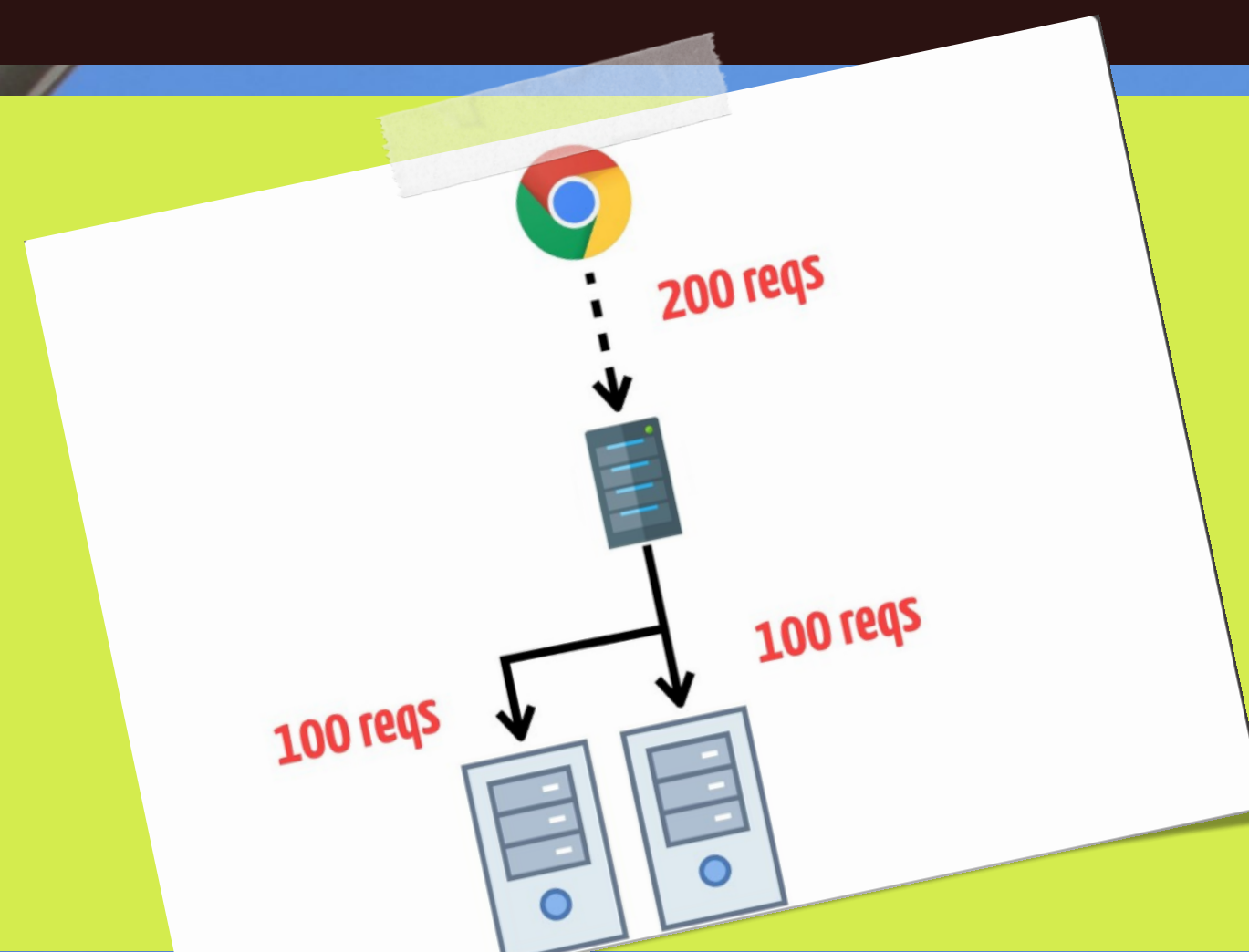
CACHE

PROCESSAMENTO
ASSINCRONO

BALANCEAMENTO
DE CARGA

CACHE

PROCESSAMENTO ASSINCRONO

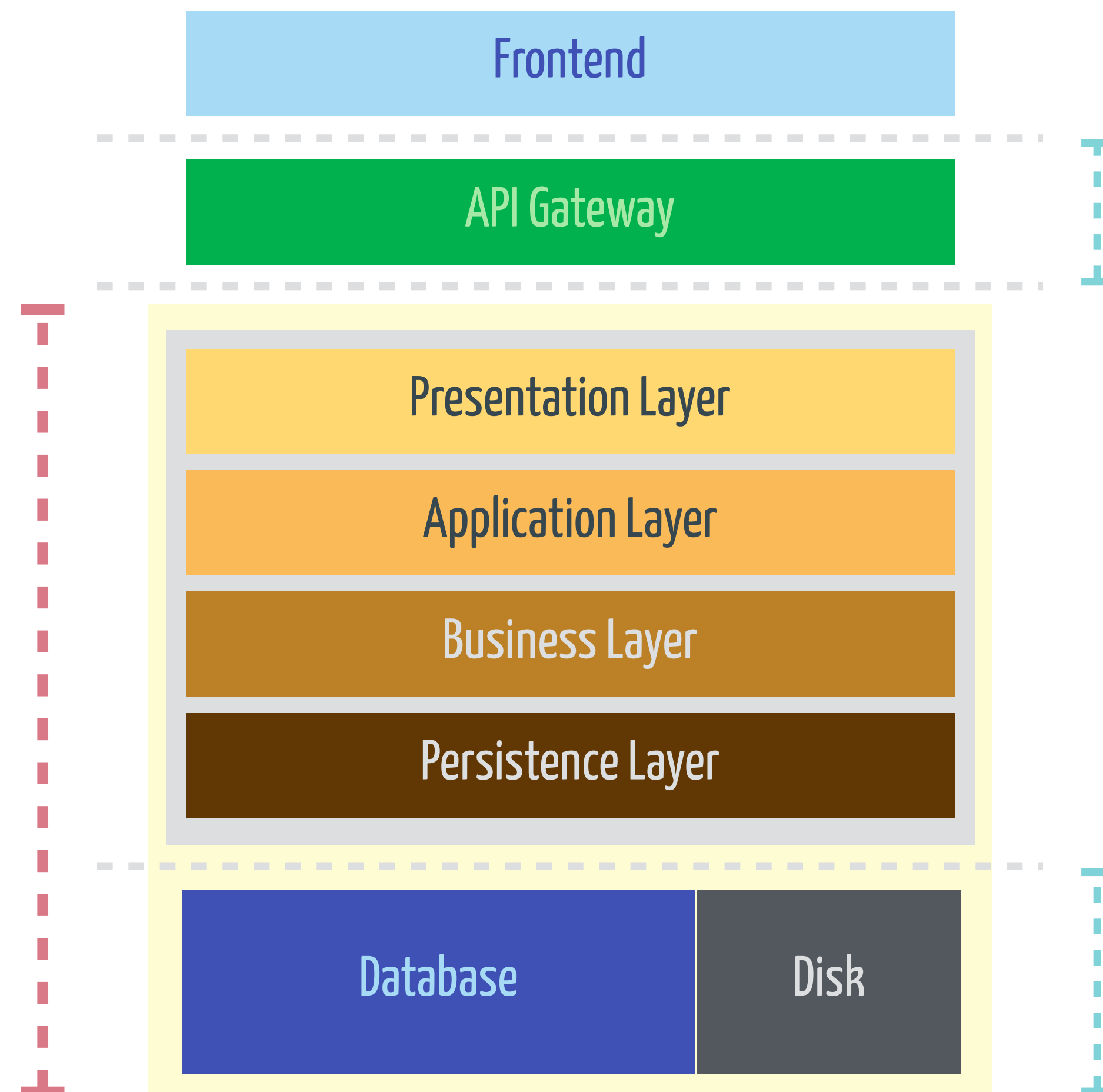


BALANCEAMENTO DE CARGA

CACHE

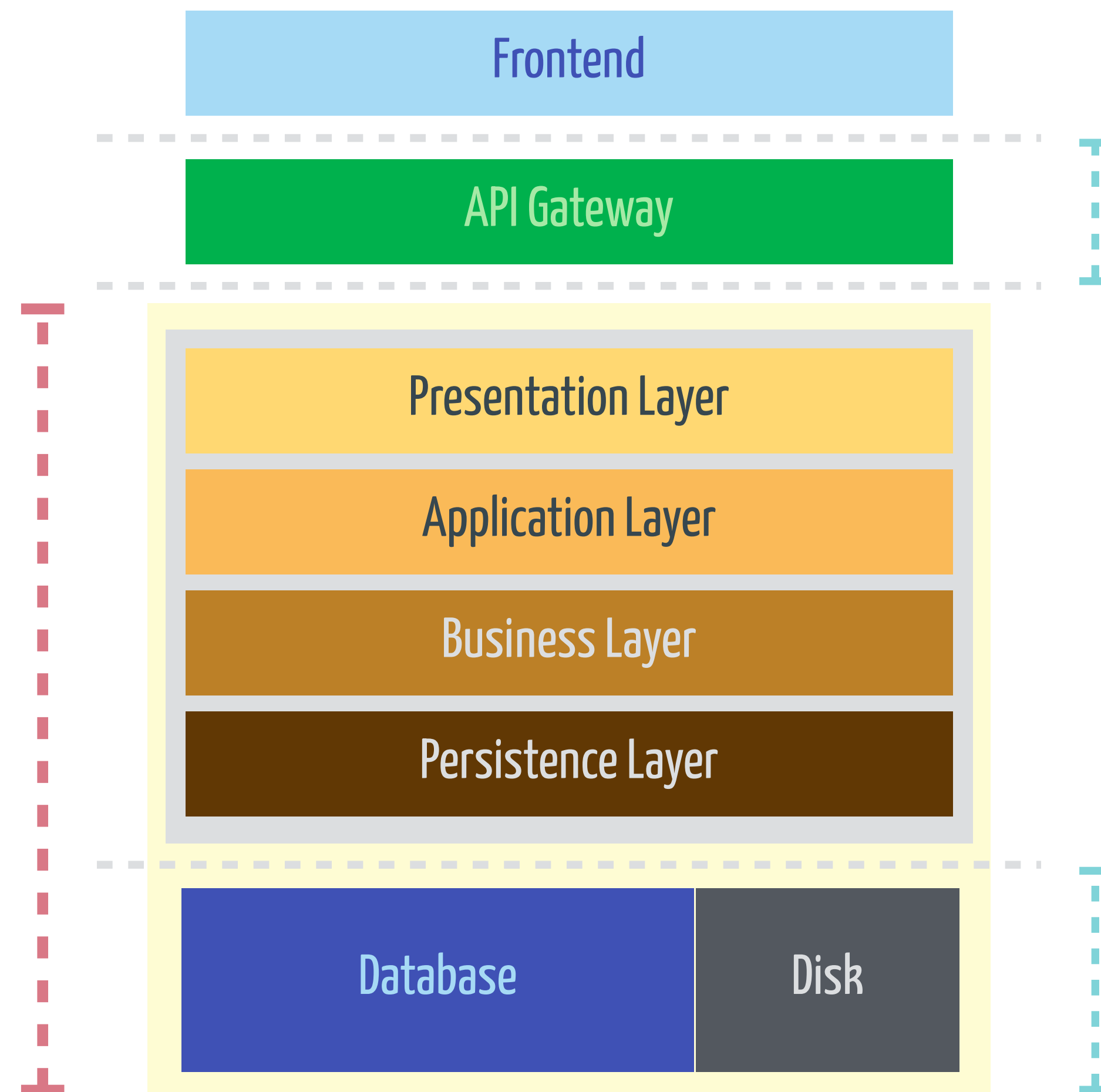
PROCESSAMENTO
ASSINCRONO

BALANCEAMENTO
DE CARGA



CACHE





In-Memory Cache

Simple and naive Cache Implementation

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

```
@Component
public class CacheStore {

    private static final Map<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = new ConcurrentHashMap<>();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

```
private static final Map<String, ExpensiveObject> cache;
```

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

`import java.util.concurrent.ConcurrentHashMap;` Simple and naive Cache Implementation

`@Component`

`public class CacheStore {`

`private static final Map<String, ExpensiveObject> cache;`

`@PostConstruct`

`public void initCache() {`

`// logic to create the cache and warm it up`

`this.cache = new ConcurrentHashMap<>();`

`}`

`public ExpensiveObject get(String key) {`

`ExpensiveObject value = cache.computeIfAbsent(key, (key) -> {`

`// logic to create the expensive object`

`return new ExpensiveObject(key, ...);`

`});`

`return value;`

`}`

`}`

In-Memory Cache

Google Guava Cache Implementation

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

In-Memory Cache

Google Guava Cache Implementation

```
import com.google.common.cache.Cache;
import com.google.common.cache.CacheBuilder;

public class CacheStore {

    private static final Cache<String, ExpensiveObject> cache;

    @PostConstruct
    public void initCache() {
        // logic to create the cache and warm it up
        this.cache = CacheBuilder.newBuilder()
            .maximumSize(100_000)
            .expireAfterWrite(1, TimeUnit.DAYS)
            .build();
    }

    public ExpensiveObject get(String key) {

        ExpensiveObject value = cache.get(key, () -> {
            // logic to create the expensive object
            return new ExpensiveObject(key, ...);
        });

        return value;
    }
}
```

Distributed Cache

PostgreSQL Materialized View Implementation

```
SELECT seller_no          as seller,  
       invoice_date       as sold_at,  
       sum(invoice_amt)   as sales_amount  
FROM invoice  
WHERE invoice_date < CURRENT_DATE  
GROUP BY  
       seller_no,  
       invoice_date;
```

```
CREATE MATERIALIZED VIEW sales_summary AS
SELECT seller_no           as seller,
       invoice_date        as sold_at,
       sum(invoice_amt)    as sales_amount
FROM   invoice
WHERE  invoice_date < CURRENT_DATE
GROUP BY
       seller_no,
       invoice_date;
```

```
CREATE UNIQUE INDEX sales_summary_seller  
ON sales_summary (seller, sold_at);
```

Para executar a View...

```
SELECT s.seller,  
       s.sold_at,  
       s.sales_amount  
FROM   sales_summary s  
WHERE  s.sold_at >= CURRENT_DATE - 7  
ORDER BY  
       s.sold_at;
```

```
SELECT s.seller,  
       s.sold_at,  
       s.sales_amount  
FROM   sales_summary s  
WHERE  s.sold_at >= CURRENT_DATE - 7  
ORDER BY  
       s.sold_at;
```

E para atualizar a View...

Distributed Cache

Postgres Materialized View
Implementation

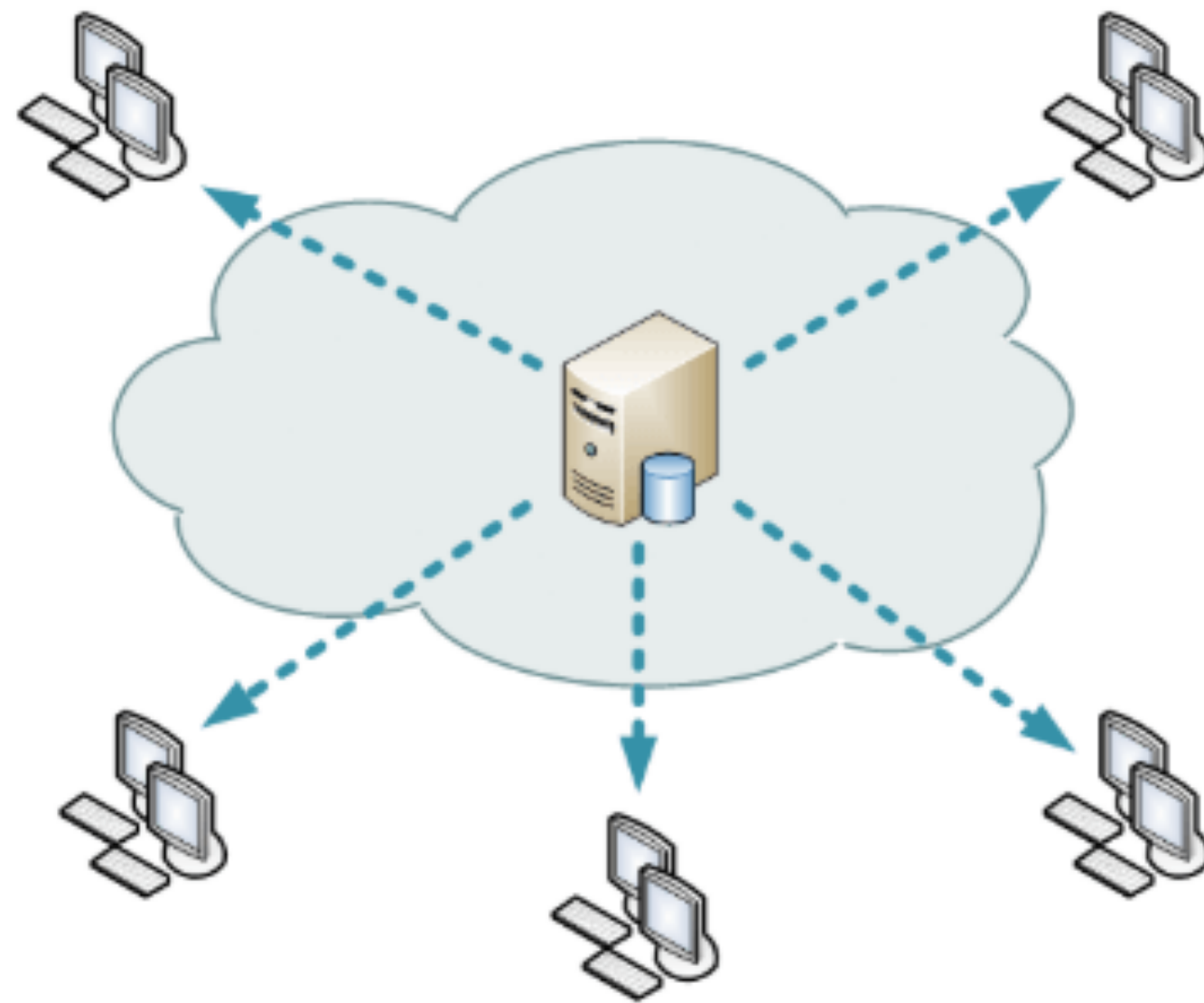
```
REFRESH MATERIALIZED VIEW sales_summary;
```

```
REFRESH MATERIALIZED VIEW CONCURRENTLY sales_summary;
```

Distributed Cache

Content Delivery Network (CDN) Implementation

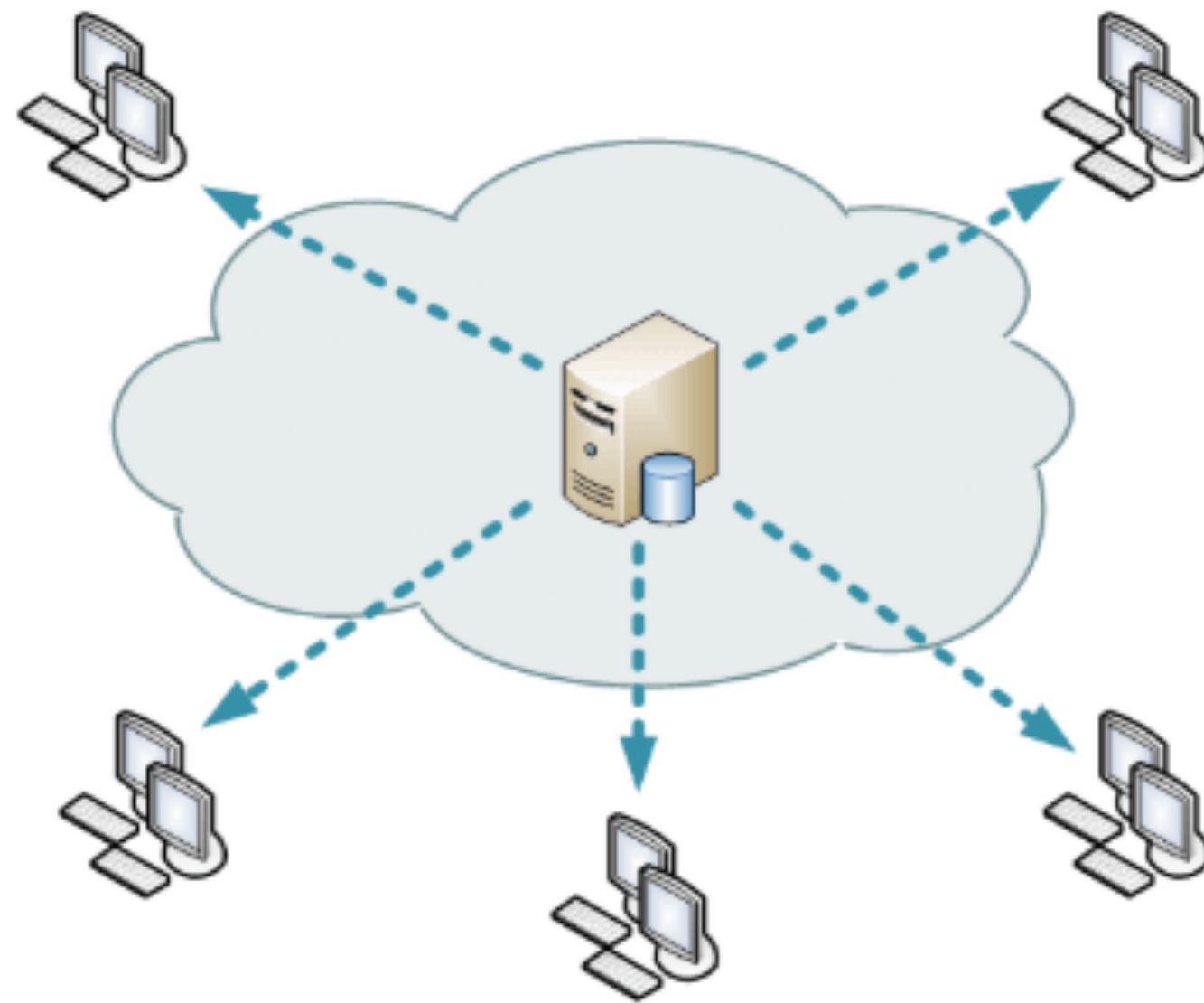
Before CDN



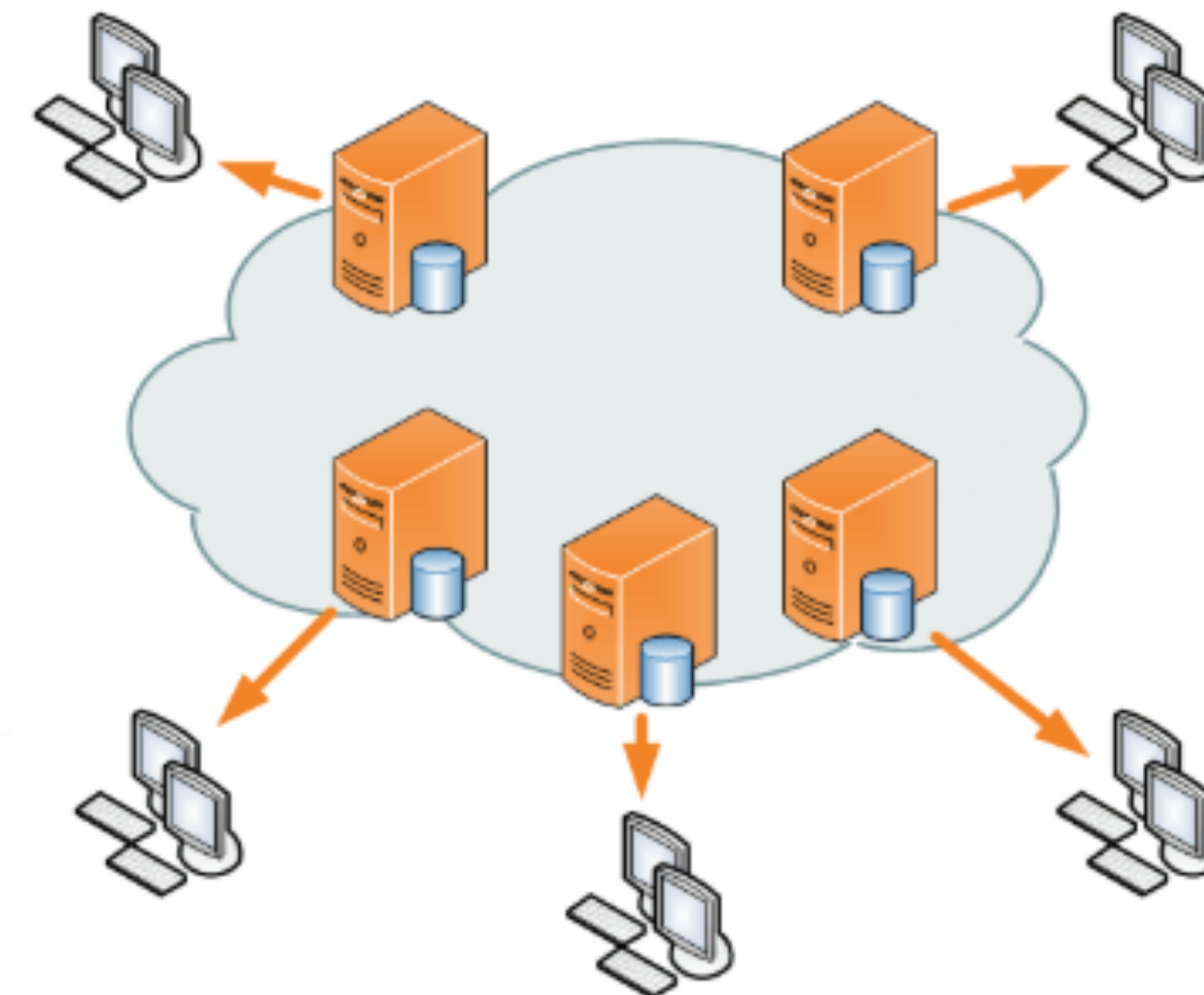
Distributed Cache

Content Delivery Network (CDN)
Implementation

Before CDN



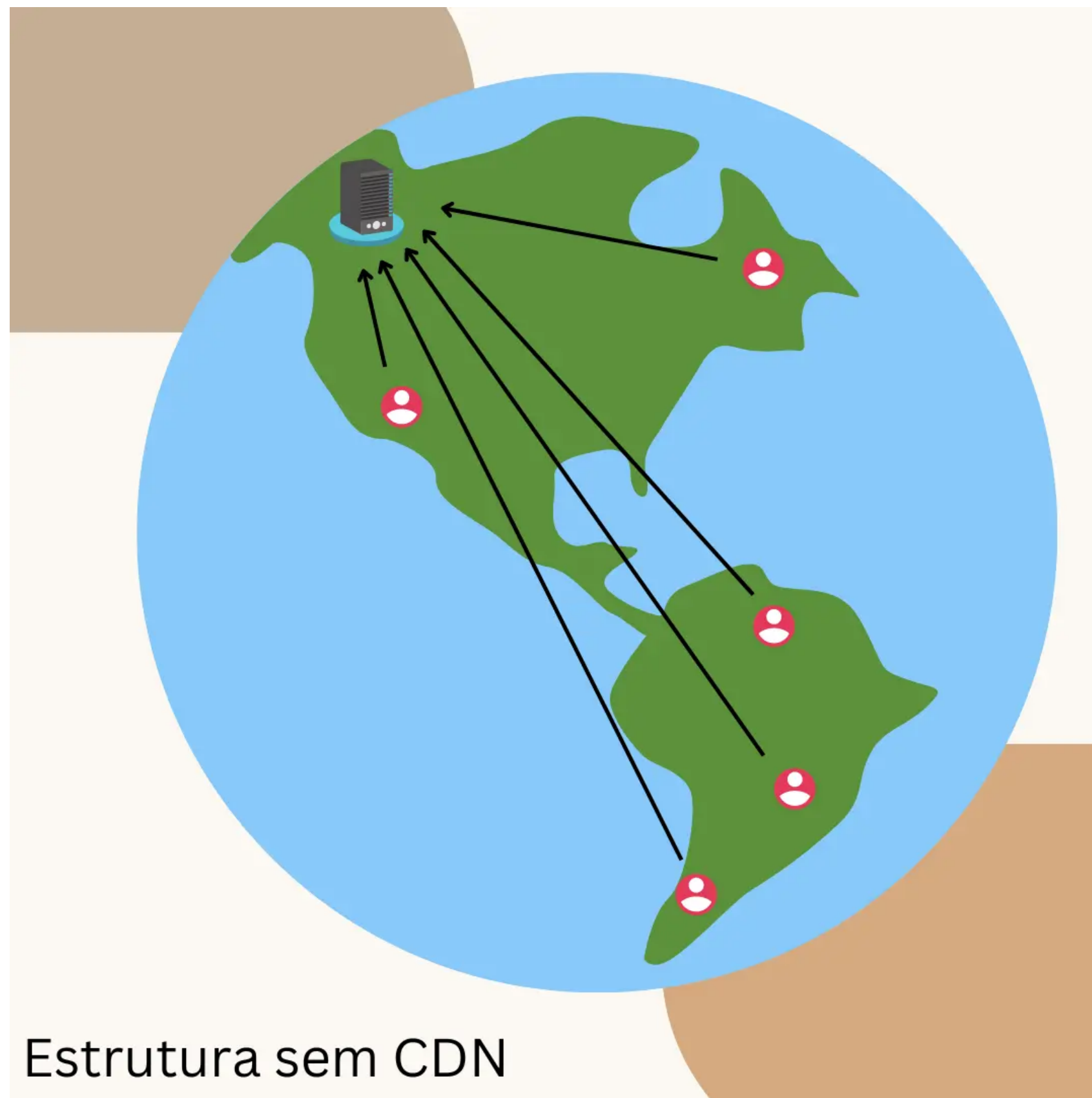
After CDN



<https://onepagezen.com/cloudflare-cdn-wordpress-google-cloud/>

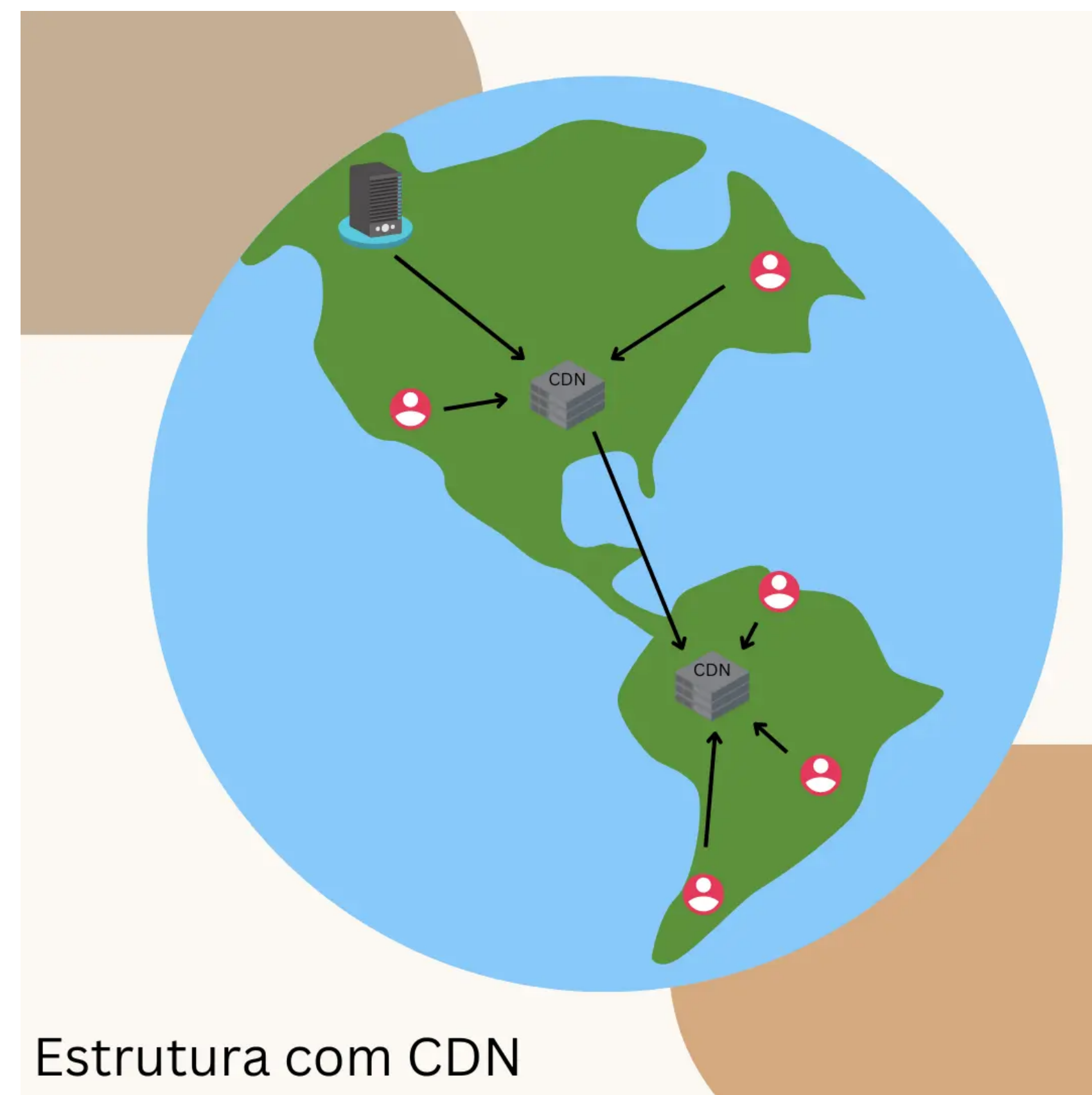
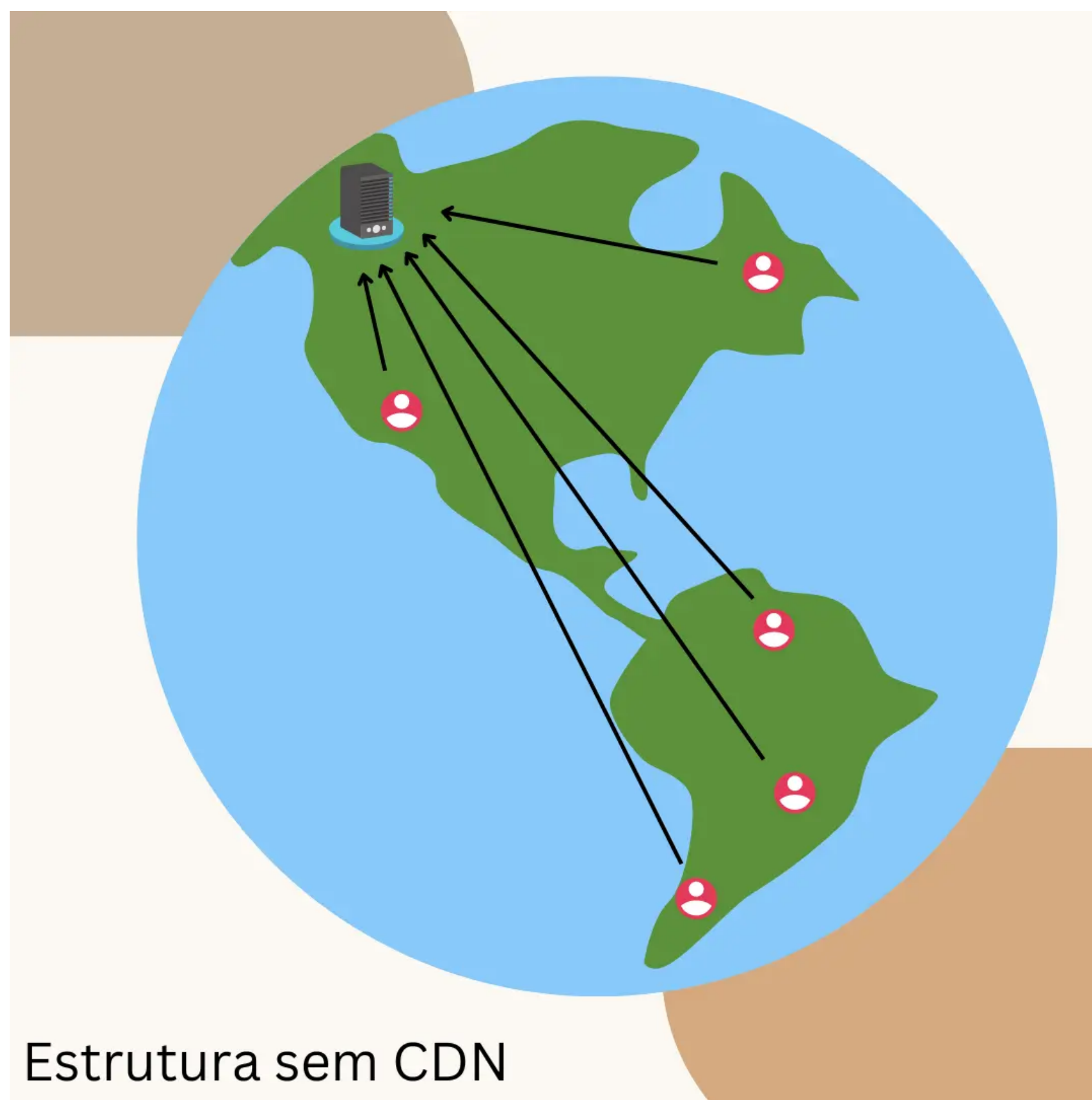
Distributed Cache

Content Delivery Network (CDN)
Implementation



Distributed Cache

Content Delivery Network (CDN)
Implementation



<https://blog.fellyph.com.br/geral/como-funciona-cloudflare-cdn/>

